



Linux SPI Driver Porting for MM610x / MM810x Wi-Fi HaLow

A complete guide for porting the Morse Micro Linux kernel driver to a new host platform using the SPI host interface

Table of Contents

Table of Contents	3
1 Scope	5
2 System Overview	6
2.1 SPI as a Host Interface	6
2.2 SPI-over-SDIO Protocol	6
3 Hardware Interface	7
3.1 Pin Mapping	7
3.2 Electrical Requirements	7
3.3 SPI Bus Timing	8
3.4 Reset Sequencing	10
4 SPI Controller Requirements	11
4.1 Transaction Size Limits	11
5 Device Tree Bindings	12
5.1 Compatible Strings	12
5.2 Required Properties	12
5.3 Optional Properties	12
5.4 Example Device Tree Node	12
6 Kernel Configuration	14
6.1 Morse Driver Kconfig Options	14
6.2 Required Kernel Subsystems	14
7 Module Parameters (Bring-Up Tuning)	15
8 Bring-Up Sequence	16
8.1 Step-by-Step Probe Sequence	16
Step 2 – SPI Controller Setup	16
Step 3 – Driver Structures and DMA Buffer	16
Step 4 – Device Tree GPIO Parse	16
Step 5 – Hardware Reset	16
Step 6 – 74-Clock Training Sequence (skipped on RK3288)	16
Step 7 – CMD63 SPI Mode Switch	16
Step 8 – Chip Detection	16
Step 9 – Inter-Block Delay Configuration	16
Step 10 – Firmware Load	16
Step 11 – IRQ Setup	16
Step 12 – MAC Registration	17
8.2 Command Protocol Details	17

8.2.1 0xFF Byte Before Each Command	17
8.2.2 0xFF Padding After Commands	17
8.2.3 0xFF Padding for CMD53 Data Phase	17
8.3 Multi-Block Transfers	17
9 Frequency Selection Guidance	19
10 Tips and Tricks	20
11 Platform-Specific Notes	21
Rockchip RK3288	21
NXP i.MX (i.MX 6 / 7 / 8)	21
TI AM335x / AM57x (McSPI)	21
Controllers with Long First-Byte Post-CS Delay	21
Any Controller Without 8-bit BPW Support	21
12 Known Issues and Workarounds	22
13 Troubleshooting	23
13.1 Verify the Driver Loaded and Matched	23
13.2 Increase Verbosity	23
13.3 Common Error Messages	23
13.4 Physical Layer Sanity Checks	23
13.5 GPIO Sanity Checks	24
13.6 Confirm the Clock Actually Used	24
13.7 Bus Integrity Test	24
Appendix A — Key Specifications	25

1 Scope

This application note guides a software engineer through every step required to port the Morse Micro Linux kernel driver to use the SPI host interface on MM610x and MM810x Wi-Fi HaLow chip families.

It covers:

- Hardware pin mapping, electrical requirements, and SPI bus timing constraints
- SPI controller capabilities required on the host
- Device tree bindings and kernel configuration
- Module parameters for bring-up tuning
- The complete bring-up sequence as executed by `morse_spi_probe()`
- Low-level SDIO-over-SPI protocol details (CMD63 fast-init, CMD53 data transfers, inter-block padding)
- Platform-specific notes for common host SoCs
- Known issues, workarounds, and a debug checklist

Readers should already be familiar with the Linux SPI subsystem (`drivers/spi/`), device-tree SPI node configuration, and the basics of the Morse Micro driver source tree. For SDIO-based porting, refer to the standard Morse Micro SDK integration guide.

2 System Overview

2.1 SPI as a Host Interface

The MM610x / MM810x SoC integrates a single physical host-interface block that implements both an SDIO 2.0 slave and a SPI slave on the same physical pins. After powering on, the block is in SDIO mode. The host switches it to SPI mode by sending CMD63 with the chip-select asserted. Once in SPI mode, the device stays in SPI mode until power is cycled or a hardware reset is applied.

Morse Micro chips MM610x / MM810x can only operate in SPI mode 0 (CPOL=0, CPHA=0). The driver configures the SPI controller mode, 8 bits per word, with a maximum clock of 50 MHz enforced in software. In practice, the usable frequency depends almost entirely on signal integrity; see Section 8 for frequency selection guidance. Morse Micro doesn't recommend SPI bus sharing with other peripheral devices

2.2 SPI-over-SDIO Protocol

The SPI interface is a thin wrapper around the SDIO digital block. This has a critical implication for driver porting: the device does not implement a pure SPI slave. It speaks the SDIO command/data protocol over SPI-compatible physical lines. Consequently:

- Commands are framed as SDIO CMD packets (6-byte command + R1/R4/R7 response).
- Data transfers use CMD53 read/write with 512-byte block granularity (MMC_SPI_BLOCKSIZE).
- The device's internal address space is 17-bit and is expanded to 32 bits via keyhole registers identical to those in SDIO mode.
- Data blocks are validated with CRC16-CCITT. The kernel CRC_ITU_T module must be enabled.
- Timing restrictions that apply to the SDIO block also apply in SPI mode because the same digital block is being clocked.

3 Hardware Interface

3.1 Pin Mapping

The SPI interface reuses the physical SDIO pins. The table below shows the mapping between the MM6108/MM8108 SDIO signal names, the SPI signal names, and the Linux SPI / GPIO driver property.

MM6108/MM8108	SPI Signal	Linux SPI Name	Notes
SDIO_CLK	SCLK	spi-sclk / CLK	Clock, driven by the host
SDIO_CMD	MOSI	spi-mosi / TX	Host → chip data
SDIO_D0	MISO	spi-miso / RX	Chip → host data
SDIO_D3	CS_N	spi-cs / CE	Active-low chip select
SDIO_D1	IRQ	spi-irq-gpios	Open-drain, active-low; needs pull-up on host
SDIO_D2	—	—	Unused; pull HIGH to VDDIO (10 kΩ)
RESET_N	nRESET	reset-gpios	Driver: assert low ≥80 ms, then release
IRQ_N	IRQ	spi-irq-gpios	
WAKE_UP	WAKE	power-gpios[0]	Optional power-save; host output
BUSY	BUSY	power-gpios[1]	Optional power-save; chip output

Note:

SDIO_D2 is unused in SPI mode and must be pulled to VDDIO (nominally 3.3 V) via 10 kΩ resistors on the PCB to prevent the internal SDIO logic from detecting bus-conflict events.

3.2 Electrical Requirements

All SPI signals are digital I/O operating at VDDIO (2.25 V – 3.6 V, typical 3.3 V). If the host SPI controller operates at 1.8 V logic levels, bi-directional level shifters will need to be inserted. For more detailed information, refer to “MM8108-MF15457 Hardware Design Guide”.

VDD should be kept within the recommended operating range of 3.0–3.6 V specified in the MM6108 and MM8108 datasheets. The MM8108 datasheet further requires that VDD exhibit less than 10 mV of ripple and less than 50 mV of load regulation for a 500 mA load, and that bypass capacitors be placed as close to the VDD pins as possible with short, wide traces. Refer to the application circuits and reference schematics in the MM6108 and MM8108 datasheets for the complete supply-decoupling networks.

3.3 SPI Bus Timing

The timing parameters are derived from the SDIO bus timing specification in the MM610x / MM810x datasheet. The table below lists all parameters relevant to the Linux SPI controller configuration.

The bus timing specification required for the SPI master is as follows:

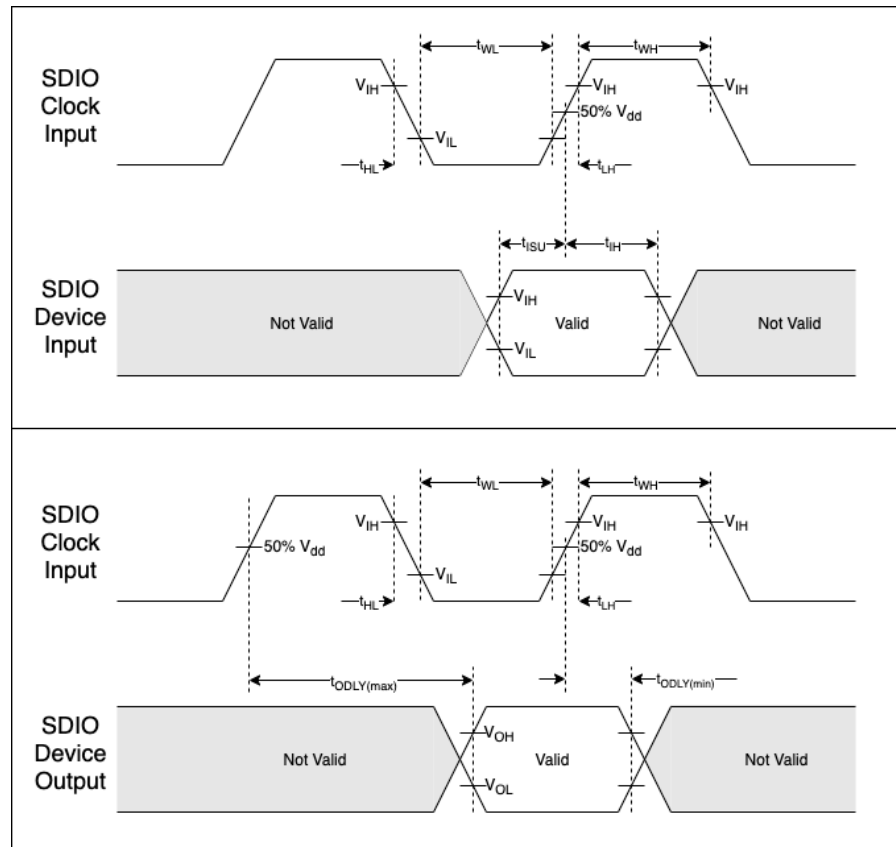


Figure 2 SDIO/SPI clock timings

Parameter	Min	Max	Notes
Clock frequency	0 MHz	50 MHz	Hard ceiling in driver (spi.c:146)

Parameter	Min	Max	Notes
CPOL / CPHA	0 / 0	0 / 0	SPI Mode 0; clock idles low, sample on rising edge
MOSI setup to SCLK rise (t_{SU})	6 ns	—	From SDIO host-mode timing spec
MOSI hold after SCLK rise (t_{IH})	2 ns	—	
SCLK rise to MISO valid (t_{ODLY})	—	14 ns	At chip pins; tightens at > 50 MHz
CS_N assert to first SCLK	0 ns	—	Keep CS asserted for full transaction
Inter-block delay	5-40 μ s	130 ms	~40 μ s for MM6108, ~5 μ s for MM8108; abort if > 130 ms
Signal capacitance	—	40 pF	Exceeding this limit requires series termination

3.4 Reset Sequencing

The driver controls nRESET via the reset-gpios device-tree property. On probe, `morse_spi_reset()` asserts nRESET low and holds it for 80 ms before releasing. This single timing covers both the RESET_N hold requirement and sufficient time for all chip power rails and the crystal oscillator to stabilise before the first SPI command.

The complete board-level power-on sequence is:

1. Assert nRESET low.
2. Ensure all chip power rails (VDD, VDDIO) are within specification, and the crystal/clock reference is running.
3. The driver will release nRESET after 80 ms. If your power ramp or crystal startup is longer than 80 ms, delay the module load accordingly.
4. After nRESET is released, the driver waits for the chip SPI state machine to enter idle, then begins the training sequence (Section 7, Step 6).

4 SPI Controller Requirements

The driver calls `spi_setup()` with the following fixed settings:

- mode: `SPI_MODE_0` (`CPOL=0, CPHA=0`) — recommended
- bits_per_word: 8 — mandatory; 16- or 32-bit modes are not supported
- Max clock: 50 MHz — hard ceiling clipped in `morse_spi_setup()` (`spi.c:202`)

Additional controller requirements and recommendations:

- A DMA-capable controller is strongly recommended. The driver issues single transactions up to 8 KB, and total transfers up to 64 KB.
- CS must remain asserted (low) for the duration of each transaction. The driver uses a single shared 8 KB DMA buffer for both TX and RX. Do not configure the controller to release CS between words or between DMA segments.
- During the 74-clock training sequence, the driver temporarily asserts `SPI_CS_HIGH` to clock out 18 bytes of `0xFF` with CS deasserted. If the controller cannot honour this, the driver logs "can't change chip-select polarity" and skips the sequence. This may affect the future operation of the chip.

4.1 Transaction Size Limits

- `SPI_MAX_TRANSACTION_SIZE` = 8 KB — maximum single SPI transaction
- `SPI_MAX_TRANSFER_SIZE` = 64 KB — maximum single bus read/write
- `MMC_SPI_BLOCKSIZE` = 512 B — SDIO block size

If your SPI controller cannot sustain 8 KB bursts without inserting long interblock gaps, throughput will suffer, but the link will remain functional.

5 Device Tree Bindings

5.1 Compatible Strings

- "morse,mm610x-spi" — for MM610x family
- "morse,mm810x-spi" — for MM810x family (includes MM8108)

The driver's OF match table is in spi.c:192 and its non-DT SPI id table is in spi.c:184.

5.2 Required Properties

- compatible — as above
- reg — chip-select index on the parent SPI bus (not a memory address)
- spi-max-frequency — SPI bus clock in Hz; must be ≤ 50000000
- reset-gpios — single GPIO specifier for nRESET (active-low). Probe fails if absent (unless mmc-pwrseq is used; see Section 9)
- spi-irq-gpios — single GPIO specifier for the chip IRQ output (active-low). Probe fails if absent

5.3 Optional Properties

- power-gpios — pair of GPIO specifiers (wake, busy) as a single property with two cells. Required only for power-save support. If only one GPIO is listed, the driver logs a warning and disables power-save
- interrupts / interrupt-parent — not required; the driver calls gpio_to_irq() on spi-irq-gpios internally

5.4 Example Device Tree Node

A complete working example for the RPi Broadcom SPI host:

```
/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2835", "brcm,bcm2836", "brcm,bcm2708",
    "brcm,bcm2709", "brcm,bcm2711";

    fragment@0 {
        target = <&spi0>;
        frag0: __overlay__ {
            pinctrl-0 = <&spi0_pins &spi0_cs_pins>;
            cs-gpios = <&gpio 8 1>;
            #address-cells = <1>;
            #size-cells = <0>;
            status = "okay";
        }
    }
}
```

```

mm6108: mm6108@0 {
    compatible = "morse,mm610x-spi";
    reg = <0>; /* CE0 */
    reset-gpios = <&gpio 5 0>;
    power-gpios = <&gpio 3 0>, <&gpio 7 0>;
    spi-irq-gpios = <&gpio 25 0>;
    spi-max-frequency = <50000000>;
    status = "okay";
};

spidev0: spidev@0 {
    reg = <0>; /* CE0 */
    status = "disabled";
};

spidev1: spidev@1 {
    reg = <1>; /* CE1 */
    status = "disabled";
};
};

fragment@1 {
    target = <&gpio>;
    __overlay__ {
        spi0_cs_pins: spi0_cs_pins {
            brcm,pins = <8>;
            brcm,function = <1>; /* BCM2835_FSEL_GPIO_OUT */
            brcm,pull = <2>; /* SET SPI PINS AS PULL HIGH */
        };
        spi0_pins: spi0_pins {
            brcm,pull = <2 2 2>; /* SET SPI PINS AS PULL HIGH */
        };
    };
};
};

```

Notes:

If operating in Mode 0, do not set spi-cpol or spi-cpha in your DT node. Mode 0 is the correct default. Setting either flag selects a different SPI mode, and the chip will not respond.

SDIO_D2 pull-ups should be placed on the PCB, not controlled by software. Confirm the schematic includes 10 k Ω pull-up resistors on these two lines.

6 Kernel Configuration

6.1 Morse Driver Kconfig Options

- `CONFIG_MORSE_SPI=y` (or `=m`) — builds the SPI bus support. Required.
- `CONFIG_MORSE_SPI_RK3288=y` — Rockchip RK3288 platforms only (see Section 11).
- `CONFIG_MORSE_DEBUG_MASK=<n>` — debug verbosity bitmask: 1=debug, 2=info, 4=warning, 8=error (default). Set to 15 (0x0F) during bring-up for maximum verbosity.
- `CONFIG_MORSE_ENABLE_TEST_MODES=y` — enables the `test_mode` module parameter used by the bus integrity test (Section 12).

6.2 Required Kernel Subsystems

- `CONFIG_SPI` — SPI master framework
- `CONFIG_OF` — device tree support
- `CONFIG_OF_GPIO` — GPIO bindings used by `morse_of_probe()`
- `CONFIG_CRC_ITU_T` — CRC16-CCITT used by the SPI data integrity check

7 Module Parameters (Bring-Up Tuning)

All parameters may be passed on the insmod / modprobe command line or persisted in /etc/modprobe.d/morse.conf using the options directive.

Parameter	Default	Description
spi_clock_speed	0 (use DT)	Override spi-max-frequency from the device tree. Values above 50 MHz are clipped.
spi_inter_block_delay_bytes	auto	Override auto-calculated inter-block delay bytes. Raise if CRC errors occur.
spi_post_write_status_bytes	4	Additional 0xFF cycles after CMD53 write for chip response token. Raise to 8 or 16 on older kernels.
spi_use_edge_irq	0	0 = IRQF_TRIGGER_LOW (level). 1 = IRQF_TRIGGER_FALLING (edge). Use the edge only if the host cannot level-trigger.
enable_ext_xtal_init	0	Set to 1 on boards booting from a slow (e.g., 32 kHz) external crystal. Adds +2 KB 0xFF padding to writes.

Examples:

```
# insmod - full set of bring-up overrides
insmod morse.ko spi_clock_speed=5000000 spi_use_edge_irq=1
enable_ext_xtal_init=1

# modprobe.d - persist a 25 MHz clock
echo "options morse spi_clock_speed=25000000" >>
/etc/modprobe.d/morse.conf
```

8 Bring-Up Sequence

This section describes exactly what happens when `morse_spi_probe()` runs (`spi.c:1387`). You can correlate kernel log output with each phase.

8.1 Step-by-Step Probe Sequence

Step 1 — Compatible / ID Match

The driver matches "morse,mm610x-spi" or "morse,mm810x-spi" from the DT, or "mm610x-spi" / "mm810x-spi" from the non-DT SPI id table, and selects the chip series.

Step 2 — SPI Controller Setup

Calls `morse_spi_setup()`: sets Mode 0, 8 bpw, clips clock to 50 MHz, and calls `spi_setup()`. Failure here means the host controller rejected the settings.

Step 3 — Driver Structures and DMA Buffer

Allocates an 8 KB buffer used as a shared TX/RX DMA region. Initialises the `spi_message` / `spi_transfer` structures used for every transaction.

Step 4 — Device Tree GPIO Parse

`morse_of_probe()` reads `reset-gpios`, `spi-irq-gpios`, and optional `power-gpios`.

Step 5 — Hardware Reset

Calls `morse_spi_reset()`: asserts `nRESET` low, waits 80 ms, releases).

Step 6 — 74-Clock Training Sequence (skipped on RK3288)

Temporarily asserts `SPI_CS_HIGH` and clocks out 18 bytes of `0xFF` (≥ 74 clock pulses) with `CS` deasserted to satisfy the MMC/SD requirement before `CMD0`. Then restores `CS` polarity.

Step 7 — CMD63 SPI Mode Switch

Issues `CMD63` (`SD_IO_MORSE_INIT`) — the vendor-specific command that switches the chip from `SDIO` mode to `SPI` mode. On failure, falls back to `CMD0` (`SD_IO_RESET`) and retries. Up to 3 attempts total. If all attempts fail, logs "failed to init SPI with `CMD63` (ret:<N>)".

Step 8 — Chip Detection

Reads the chip ID register and validates that it matches the compatible string. On success logs "Morse Micro SPI device found, chip ID=0x<NNNN>". A return of all `0xFF` or all `0x00` indicates the initialisation sequence failed; see Section 13.

Step 9 — Inter-Block Delay Configuration

Configures inter-block timing based on clock speed (or `spi_inter_block_delay_bytes` if overridden). On success logs "clock=<N> MHz, delay bytes=<N>, max block count=<N>".

Step 10 — Firmware Load

Loads the firmware image. Out of scope for this note; see the Morse Micro firmware application note.

Step 11 — IRQ Setup

Calls `morse_spi_setup_irq()`:

- `gpio_request()` and `gpio_direction_input()` on `spi-irq-gpios`

- `gpio_to_irq()` to obtain the Linux IRQ number
- CMD52 writes to SDIO CCCR registers 0x04 (IEN) and 0x07 (BIC) to enable chip-side interrupts
- `request_threaded_irq()` with `IRQF_TRIGGER_LOW | IRQF_ONESHOT` (or `IRQF_TRIGGER_FALLING` if `spi_use_edge_irq=1`)

Step 12 — MAC Registration

Registers the 802.11 MAC layer and starts the health check thread. Beyond this point, the SPI link is up, and the driver is in the MAC domain.

Note:

If the probe fails, the driver reverses each step in order, unless the chip supports the fast re-probe (reattach) path that keeps firmware loaded.

8.2 Command Protocol Details

8.2.1 0xFF Byte Before Each Command

Each SDIO command must be immediately preceded by a single 0xFF byte sent with CS asserted. This is the mandatory 'clock-before-command' byte required by the SDIO-over-SPI protocol, so the chip can detect the command start bit (0b). Omitting it causes intermittent failures, especially on USB-to-SPI bridges with non-deterministic inter-transaction timing.

8.2.2 0xFF Padding After Commands

After transmitting an SDIO command (while CS is still asserted), include one byte of 0xFF in the same SPI transfer message before polling for the device response. Sending it as a separate transfer can introduce enough delay that MISO takes significantly longer to return high.

8.2.3 0xFF Padding for CMD53 Data Phase

For CMD53 read/write operations, up to four additional 0xFF bytes may be required between the command status response and the data token (0xFE for single block, 0xFC for multi-block write). The driver must keep sending 0xFF and monitoring MISO for the data start token within this window.

8.3 Multi-Block Transfers

Multi-block transfers require special handling because the chip needs inter-block time to process received data:

- After each block's status/acknowledgement, continue sending 0xFF on MOSI until MISO returns high (chip not busy).
- If MISO has not returned high within 130 ms, the block transfer has failed — abort and retry from the start of the CMD53 sequence.
- The entire multi-block transfer (command + all blocks + padding) must be a single SPI transaction with CS asserted throughout. Toggling CS between blocks causes the chip to lose block-boundary tracking.

- As the interblock delay is a constant value, the number of 0xFF inter-block padding bytes varies with clock speed. At lower clocks (e.g., 12.5 MHz), fewer bytes are needed than at 50 MHz.

9 Frequency Selection Guidance

The absolute maximum SPI clock the driver accepts is 50 MHz. In practice, the usable frequency depends almost entirely on signal integrity: trace length, trace matching, connector quality, and the host SPI controller's output slew rate.

Recommended bring-up strategy:

5. First boot: 1–5 MHz — at this speed, electrical issues are eliminated as a cause of failure. Start here.
6. Functional test: 10–20 MHz — once chip ID reads back correctly and firmware loads, step up to a mid-range clock. Run iperf or the bus test mode (Section 12) for several minutes and watch for CRC errors in dmesg.
7. Target operation: 20–25 MHz — the sweet spot for most production designs.
8. Above 25 MHz — possible on well-engineered PCBs with short length-matched traces, clean grounds, and controllers with fast output slew. Validate with an oscilloscope on MISO/MOSI/SCLK at the chip pins.

Inter-block delay tuning: the default inter-block delay (40 μ s for MM6108 and 5 μ s for MM8108) is expressed internally as a count of 0xFF padding bytes at the configured clock. If you see sporadic "SPI response missing" or CRC errors at a given clock, raise `spi_inter_block_delay_bytes` before dropping the clock.

Note:

Reducing the clock speed does not always improve reliability. At very low clocks (< 12.5 MHz), the inter-block gap is proportionally shorter, and the number of required padding bytes decreases. If debugging SPI issues, test at both the configured speed and at 25 MHz before concluding the issue is clock-rate-independent.

10 Tips and Tricks

- reset-gpios and spi-irq-gpios must be on host pins that have no bootstrap or alternate function that could drive them during reset. A bootstrap pin forced high can prevent the chip from releasing reset or mask the chip IRQ.
- The chip IRQ (IRQ_N) is an open-drain, active-low line. Ensure the host input is not configured with an internal pull-down — the interrupt will never deassert.
- The IRQ line should be routed to a GPIO that supports wake-from-suspend if system-wide power management is required.
- If using mmc-pwrseq to control power and reset instead of reset-gpios, the driver tolerates a missing reset-gpios property and logs an info message rather than failing probe (of.c:33).
- On boards with a 32 kHz sleep oscillator as the only clock reference at boot, set enable_ext_xtal_init=1. The driver will pad every write with +2 KB of 0xFF bytes to give the chip time to respond at the slow clock rate.
- The driver's DMA buffer is a single 8 KB buffer used for both TX and RX. CS must be held for the full transaction; do not configure the controller to release CS between words.
- When swapping between SDIO and SPI on the same hardware, remember SPI requires both reset-gpios and spi-irq-gpios; SDIO requires neither.
- For development, always start with spi_clock_speed lower than your DT value. Once the link is stable, remove the override and rely on spi-max-frequency.

11 Platform-Specific Notes

Raspberry Pi (BCM2835 / 2837 / 2711)

Works out of the box at 25 MHz with the standard spi0 or spi1 master. Use a DT overlay to add the morse_wifi@0 node. No special Kconfig options are required.

Rockchip RK3288

Requires CONFIG_MORSE_SPI_RK3288=y. The driver skips the 74-clock CS-high training sequence (the controller cannot drive CS high) and applies a 1-bit right-shift to all received data to compensate for a known controller off-by-one bug.

NXP i.MX (i.MX 6 / 7 / 8)

Works at Mode 0 on all ECSPI variants. Do not set spi-cpol or spi-cpha in your DT — the defaults (both 0) are correct. The ECSPI maximum usable clock is typically well below 50 MHz, so the driver ceiling is not the limiting factor.

TI AM335x / AM57x (McSPI)

Check the McSPI cs-word / cs-release behaviour. The driver requires CS held for the full transaction; verify the controller does not release CS between DMA segments.

Controllers with Long First-Byte Post-CS Delay

Some controllers insert gaps after asserting CS. Raise spi_post_write_status_bytes from 4 to 8 or 16.

Any Controller Without 8-bit BPW Support

Not supported. The driver requires 8 bits per word.

12 Known Issues and Workarounds

Symptom	Cause	Workaround
All MISO bytes are 0xFF after CMD63	CMD63 not accepted; chip still in SDIO mode	Verify 18-byte 0xFF training with CS high was sent; confirm reset timing (≥ 80 ms); check SDIO_D1/D2 pull-ups
MISO data shifted by exactly 1 bit	Unexpected chip reset (JTAG_RST, supply drop, GPIO glitch)	Ensure JTAG_RST is not driven by host tools; check VDD stays above 3.0 V; driver must detect and re-init
Intermittent CMD53 read failures	Insufficient inter-block padding or single-transaction violation	Raise spi_inter_block_delay_bytes; ensure entire multi-block transfer uses a single CS assertion
Truncated / corrupted reads at < 25 MHz	Inter-block gap longer at low clock; DMA buffer too small	Increase driver DMA receive buffer; add extra 0xFF padding to single-block reads
"SPI response missing" errors	Chip not emitting R1 token in the expected window	Raise spi_post_write_status_bytes (try 8, then 16); reduce clock speed
FTDI JTAG_RST assertion on connect	FT2232H D2XX driver asserts ADBUS7 (JTAG_RST) at init	Disconnect JTAG_RST or deassert ADBUS7 before SPI communication begins
"can't change chip-select polarity"	SPI controller cannot drive CS high for the training burst	Informational only; driver proceeds. If CMD63 also fails, emit 74 clocks externally or use the RK3288 workaround
CRC errors above 25 MHz	Signal integrity: long traces, capacitance, slew mismatch	Add 22–33 Ω series terminators near the chip; reduce the clock; validate with a scope at chip pins

13 Troubleshooting

13.1 Verify the Driver Loaded and Matched

Run `dmesg | grep -i morse` and look for:

```
"Reading gpio pins configuration from device tree"
```

Then follow the step-by-step messages from Section 8 to identify which probe step failed.

13.2 Increase Verbosity

Build with `CONFIG_MORSE_DEBUG_MASK=0xF` or override at runtime via the driver's module parameter `debug_mask=0xF`.

13.3 Common Error Messages

- "Required property reset-gpios not found in device tree" → Add reset-gpios to your DT node. Verify the loaded DTB with `/proc/device-tree/`.
- "Required property spi-irq-gpios not found in device tree" → Add spi-irq-gpios to your DT node.
- "Failed to acquire spi irq gpio" → Another driver already claimed the GPIO, or the GPIO number is invalid. Check `/sys/kernel/debug/gpio`.
- "failed to init SPI with CMD63 (ret:<N>)" → Chip not responding to the init command. Verify SPI wiring, power rails, crystal, nRESET timing (≥ 80 ms), SPI Mode 0, and try `spi_clock_speed=1000000` to rule out signal integrity.
- "SPI response missing" → Chip not emitting R1 response. Try raising `spi_post_write_status_bytes` (e.g., 8), then `spi_inter_block_delay_bytes`. If persistent, lower the clock.
- "SPI response bit shifted" → MISO sampled one bit early/late. On RK3288, fixed by `CONFIG_MORSE_SPI_RK3288=y`. On other controllers, check CPHA and controller RX sample-delay settings.
- "SPI response error" → R1 byte invalid; likely noise or bad CRC. Lower the clock and scope MISO at the chip pins.
- "can't change chip-select polarity" → Informational; controller does not support `SPI_CS_HIGH`. Training sequence is skipped. Usually harmless.
- "SPI clock <N> MHz is not supported by this chip, clipping to 50 MHz" → Your DT or module param exceeds 50 MHz; the driver clips it.

13.4 Physical Layer Sanity Checks

With a scope or logic analyser on the SPI pins measured at the chip:

- Confirm SCLK idles low and samples on the rising edge (Mode 0).
- Confirm CS is driven low for the entire transaction, not just the first byte.

- Confirm the 18-byte 74-clock CS-high training burst appears on MOSI with CS high (absent on RK3288 targets).
- Confirm the first CMD bytes match SDIO framing (0x40 | cmd_index on MOSI).

13.5 GPIO Sanity Checks

Use `gpioinfo` (`libgpiod`) to verify:

- `reset-gpios`: output, currently high after reset release.
- `spi-irq-gpios`: input, currently high when idle (chip releases line when no IRQ pending). Verify no internal pull-down on the host input.
- `power-gpios wake`: output, host-controlled.
- `power-gpios busy`: input, chip-driven.

13.6 Confirm the Clock Actually Used

Check `dmesg | grep "clock="`. You should see: `"clock=25 MHz, delay bytes=<N>, max block count=<N>"`. If the clock is lower than your DT setting, a `spi_clock_speed` module parameter may be overriding it.

13.7 Bus Integrity Test

This is a development-only feature and not always guaranteed to work.

Build with `CONFIG_MORSE_ENABLE_TEST_MODES=y` and load the module with `test_mode=MORSE_CONFIG_TEST_MODE_BUS`. This runs an internal pattern test over the SPI link, exercising the CRC path independent of firmware. A clean pass indicates the bus is healthy.

Use `MORSE_CONFIG_TEST_MODE_BUS_PROFILE` to measure throughput versus block size, which is useful for tuning `spi_inter_block_delay_bytes`.

Appendix A – Key Specifications

All values below are derived from #define constants in spi.c.

Parameter	Value
Maximum SPI clock	50 MHz (hard ceiling, driver clips higher values)
SPI mode	Mode 0 (CPOL=0, CPHA=0)
Bits per word	8
SDIO block size	512 bytes
Driver DMA buffer	8 KB (single shared TX/RX buffer)
Maximum single SPI transaction	8 KB
Maximum single bus read/write	64 KB
Reset assert / hold / release	Drive nRESET low, hold ≥80 ms, release
74-clock training burst	18 bytes of 0xFF with CS deasserted (CS high)
Default inter-block delay	~40 µs for MM6108 ~5 µs for MM8108 (expressed as padding bytes at clock speed)
Default post-write status bytes	4
Data integrity CRC	CRC16-CCITT
IRQ trigger (default)	IRQF_TRIGGER_LOW IRQF_ONESHOT
IRQ trigger (edge mode)	IRQF_TRIGGER_FALLING IRQF_ONESHOT

Morse Micro provides this information "as is" without warranties of any kind. Timing specifications are derived from driver source code and internal platform testing. It is the reader's responsibility to test and validate all product performance and compliance. This document contains proprietary information of Morse Micro Pty. Ltd. and is subject to change without notice.

Wi-Fi®, Wi-Fi HaLow™, and the Wi-Fi logo are trademarks of Wi-Fi Alliance.

Morse Micro Pty. Ltd. | Level 8, 10-14 Waterloo Street, Surry Hills NSW 2010, Australia | sales@morsemicro.com

Copyright © Morse Micro Pty. Ltd.